

DAE5 Data Engineering: Semesterprojekt

Projektbeschreibung · WS 2026/27 · FH OÖ Campus Hagenberg · Version 1.3 · Stand 30.09.2026

Kurzfassung in drei Sätzen.

Sie entwerfen zu zweit eine kleine, aber vollständige Datenbank für einen selbstgewählten Anwendungsfall: vom konzeptuellen Modell über das normalisierte Schema bis zu lauffähigem SQL.

Anschließend vertiefen Sie **genau einen** Engineering-Aspekt Ihrer Wahl (Performance, Nebenläufigkeit, Verifikation, Zugriffskontrolle, Analytics oder analytische SQL).

Zum Abschluss bearbeitet **jede Person einzeln** ein ihr persönlich zugeteiltes

Änderungsszenario an Ihrem eigenen Modell. Dort entscheidet sich ein wesentlicher Teil Ihrer Note.

Die Lehrveranstaltung ist LV-immanent und wird vollständig schriftlich beurteilt. Es gibt keine mündliche Prüfung.

0. Inhalt

Kapitel	Inhalt
1	Worum es geht und worum ausdrücklich nicht
2	Organisatorisches: Gruppen, Termine, Umfang
3	Teil 1: BUILD (B1-B11), Pflichtteil, Gruppenleistung
4	Teil 2: ENGINEERING EXTENSION (E1-E6), genau eine Option, Gruppenleistung
5	Teil 3: ÄNDERUNGSSZENARIO, Einzelleistung
6	Abgabeformat und Repository-Struktur
7	Bewertung: Kurzüberblick
8	KI-Nutzung: Kurzüberblick
9	Häufige Fragen
10	Abgabe-Checkliste

1. Worum es geht und worum ausdrücklich nicht

1.1 Ziel

Am Ende dieses Projekts sollen Sie **eine Sache wirklich können**, statt zehn Dinge gesehen zu haben:

Von einer unscharf beschriebenen Miniwelt zu einem begründeten, normalisierten, lauffähigen und überprüfbaren relationalen Datenmodell zu kommen und jede einzelne Entscheidung darin verteidigen zu können.

Alles Weitere in dieser Lehrveranstaltung (Transaktionen, Indizes, Sicherheit, Analytics, Cloud) baut auf dieser Fähigkeit auf. Deshalb ist sie der Kern des Projekts, und deshalb ist der Rest bewusst schmal gehalten.

1.2 Was dieses Projekt *nicht* ist

Damit keine Missverständnisse entstehen: Die folgenden Dinge sind **nicht** gefordert und bringen **keine** Zusatzpunkte:

Nicht gefordert	Begründung
Eine vollständige, mehrschichtige Java-Anwendung	Das ist SWE-Stoff, nicht DAE5-Stoff. Ein minimaler Integrationsnachweis genügt (B9).
Ein Frontend / UI	Dieselbe Begründung.
Cloud-Deployment	Cloud wird in der LVA behandelt, aber nicht im Projekt geprüft.
Screenshots von Query-Ergebnissen	Werden durch <code>verify.sql</code> ersetzt (B8). Das ist überprüfbar, Screenshots sind es nicht.
BCNF um jeden Preis	3NF ist die geforderte Zielnormalform. Siehe B4.
Eine umfangreiche PDF-Dokumentation	Gefordert ist ein kurzes, präzises Entscheidungsprotokoll (B10), keine Seitenzahl.
Delete-Skripte / Aufräum-DDL	Wird über einen reproduzierbaren Neuaufbau gelöst, nicht über händisches Löschen.
Umfangreiche Views- und Rechte-Konstrukte	Nur relevant, wenn Sie Extension E4 wählen.

1.3 Bezug zum Raindrops-System

Im Unterricht arbeiten wir durchgehend mit **Raindrops** (Sensoren, Feuchtigkeitsmessungen; Code im Kurs-Codespace, Vorlage `DAE5_CodeSpace_WS2026`). Raindrops ist Ihr **Referenzsystem**: Dort sehen Sie, wie ein durchgezogenes Beispiel aussieht: Schema, DAO-Schicht, Tests, Idempotenz, Transaktionen.

Ihr Projekt ist nicht Raindrops. Sie wählen eine eigene Miniwelt. Sie dürfen sich an Raindrops orientieren, Muster übernehmen und Code als Vorbild verwenden. Sie dürfen aber nicht einfach Raindrops mit anderen Tabellennamen abgeben.

2. Organisatorisches

2.1 Gruppengröße

- **Zweiergruppen.** Keine Einzelarbeiten, keine Dreiergruppen (Ausnahmen nur bei ungerader Teilnehmerzahl, dann nach Absprache). Ihre Gruppe melden Sie bis Fr, 09.10.2026 per E-Mail an die LV-Leitung (siehe 2.3). Wer bis dahin keine Gruppe hat, meldet sich ebenfalls per E-Mail; die LV-Leitung bildet dann Restgruppen.
- Teil 1 und Teil 2 sind **Gruppenleistungen** (80 Punkte).
- Teil 3, das Änderungsszenario, ist eine **Einzelleistung** (20 Punkte). Jede Person bekommt ein **eigenes** Szenario und gibt ein **eigenes** Dokument ab.
- Daraus folgt: Beide Personen müssen das gemeinsame Modell **vollständig** verstanden haben. Es gibt keine sinnvolle Arbeitsteilung der Art „du machst das Modell, ich mache SQL“: Ihr persönliches Szenario kann jeden Teil des Modells treffen.

2.2 Zeitbudget

Diese Lehrveranstaltung hat 2,5 ECTS $\approx$ 62,5 Stunden Gesamtaufwand pro Person. Davon:

Anteil	Stunden/Person	Anmerkung
Präsenz (6 Blocktermine)	$\approx$ 21 h	

Anteil	Stunden/Person	Anmerkung
Vor-/Nachbereitung, Übungen	≈ 8 h	
Projekt: BUILD	≈ 18-22 h	Zweiergruppe, gemeinsam ca. 36-44 h
Projekt: Extension	≈ 6-8 h	Zweiergruppe
Projekt: Änderungsszenario	≈ 4-5 h	pro Person einzeln
Puffer	≈ 3 h	

Wenn Sie deutlich über diesem Budget landen, machen Sie mit hoher Wahrscheinlichkeit etwas, das nicht gefordert ist. Lesen Sie dann noch einmal Abschnitt 1.2 und fragen Sie nach.

2.3 Termine

Hinweis: Die Termine sind verbindlich. M1 bis M6 bezeichnen in diesem Dokument **Meilensteine**, nicht die gleichnamigen Foliensätze.

#	Meilenstein	Termin	Format	Bewertet
Start	Gruppe und Repository	Fr, 09.10.2026	Zweiergruppe per E-Mail an die LV-Leitung bekannt geben (beide Namen, GitHub-Benutzernamen, Arbeitstitel); Gruppen-Repository anlegen (Link im eLearning) und der LV-Leitung Zugriff geben	nein
M1	Projektidee + B1 (Anforderungen)	Fr, 23.10.2026, 23:59	Repository-Push, Rückmeldung im nächsten Termin	ja (5 P)
M2	B2 (konzeptuelles Modell) als Hausübung	vor Termin 3 (Fr, 13.11.2026)	Repository-Push, Besprechung im Termin	Feedback, Punkte fallen erst final
M3	Wahl der Extension bekanntgeben	bis Termin 5 (Fr, 04.12.2026)	eine Zeile in docs/20-extension.md	nein
M4	Ausgabe der Änderungsszenarien	letzter Präsenztermin (Termin 6, Fr, 11.12.2026)	persönliche Zuteilung, schriftlich	nein
M5	Finale Abgabe: BUILD + EXTENSION + ÄNDERUNGSSZENARIO	Fr, 08.01.2027, 23:59	Repository, Tag <code>abgabe-final</code>	ja (95 P)
M6	Korrektur und Notenbekanntgabe	bis 31.01.2027		

Warum zwei Zwischenabgaben? Weil die beiden häufigsten Gründe für ein schlechtes Projektergebnis sind: (a) eine Miniwelt, die zu groß oder zu trivial ist, und (b) ein konzeptuelles Modell, das in eine Sackgasse führt. Beides lässt sich im Oktober in zehn Minuten korrigieren und im Jänner nicht mehr.

Verspätete Abgabe M5: –10 Punkte pro angefangenem Tag. Ab 3 Tagen Verspätung gilt der Abgabetermin als versäumt (nicht bestanden), außer bei nachgewiesener Verhinderung.

Fehlendes Änderungsszenario: Ohne abgegebenes Änderungsszenario kann die LVA nicht positiv abgeschlossen werden (siehe Bewertungsrubrik, Mindestschwellen).

2.4 Themenwahl

Sie wählen Ihre Miniwelt selbst. Kriterien für eine **gute** Wahl:

✓ Passend, wenn ...

- die Miniwelt **4-6 Entitätstypen** trägt, nicht mehr,
- mindestens eine **echte N:M-Beziehung** entsteht, die sachlich begründet ist,
- mindestens **eine Generalisierung oder ein schwacher Entitätstyp** sich natürlich ergibt,
- es **funktionale Abhängigkeiten gibt, die nicht offensichtlich sind**, also welche, über die man diskutieren kann,
- Sie die Miniwelt gut genug verstehen, um Fragen zur Semantik beantworten zu können.

✗ Ungeeignet, wenn ...

- es im Kern nur „Benutzer hat viele Einträge“ ist (zu wenig Struktur),
- Sie ein ERP-System oder ein soziales Netzwerk „komplett“ modellieren wollen (zu groß),
- die Miniwelt eine 1:1-Kopie eines Lehrbuchbeispiels oder von Raindrops ist,
- Sie die Domäne selbst nicht kennen und alle Fachregeln erfinden müssen.

Beispiele für tragfähige Themen (keine Vorgabe, nur Kalibrierung): Vereins-/Kursverwaltung mit Terminen und Anwesenheiten · Bibliothek mit Exemplaren, Vormerkungen und Mahnstufen · Werkstatt mit Aufträgen, Positionen und Ersatzteilen · Musikfestival mit Bühnen, Slots und Künstlerverträgen · Laborbuch mit Proben, Messreihen und Geräten · Speiseplanung mit Rezepten, Zutaten, Allergenen und Chargen.

Ihre Themenwahl wird in M1 bestätigt. Erst danach sollten Sie mit B2 beginnen.

3. Teil 1: BUILD (Pflichtteil, Gruppenleistung)

Umfang: ≈ 18-22 h pro Person · **Punkte:** 55 von 100 (+ 10 Punkte für B8, siehe Bewertungsrubrik)

Der BUILD-Teil geht die Kette **Miniwelt → Modell → Schema → Daten → Abfragen → Nachweis** einmal vollständig durch. Jeder Schritt baut auf dem vorigen auf. Arbeiten Sie in dieser Reihenfolge.

B1: Anforderungen und Abgrenzung

Abgabe: docs/01-anforderungen.md · **Fällig:** M1 (23.10.2026)

Beschreiben Sie Ihre Miniwelt so, dass eine außenstehende Person versteht, was modelliert wird und was nicht.

Gefordert:

1. **Kurzbeschreibung der Miniwelt** (max. 1 Seite): Wer nutzt das System, wofür, welche Vorgänge bildet es ab?
2. **Fünf priorisierte User Stories** im Format
Als <Rolle> möchte ich <Ziel>, damit <Nutzen>.
mit Priorität 1-5 (1 = wichtigste). Diese fünf Stories legen fest, was Ihr Modell können muss, und sind später der Maßstab, an dem Ihre sechs SQL-Abfragen (B7) gemessen werden.
3. **Explizite Abgrenzung:** Mindestens drei Dinge, die Sie bewusst **nicht** modellieren, jeweils mit einem Satz Begründung.
Beispiel: „Bezahlvorgänge werden nicht modelliert. Das System dokumentiert nur, dass eine Rechnung gestellt wurde, nicht wie sie beglichen wird.“

4. **Fünf Fachbegriffe** Ihrer Domäne mit Definition (ein Mini-Glossar). Das zwingt Sie, Begriffe zu schärfen, bevor sie zu Entitätstypen werden.

Worauf es ankommt: Nicht Länge, sondern Schärfe. Eine Abgrenzung, die konkret benennt, was draußen bleibt, ist mehr wert als drei Seiten Prosa.

B2: Konzeptuelles Modell

Abgabe: docs/02-modell.drawio + docs/02-modell.png (oder .pdf) + docs/02-modell.md

Fällig: M2 (vor 13.11.2026) als Hausübung, finale Fassung in M5

Modellieren Sie Ihre Miniwelt als **EER-Diagramm**.

Verbindliche Vorgaben:

Vorgabe	Details
Notation	(min,max)-Notation. Keine Krähenfüße, keine gemischten Notationen. Eine Notation, konsequent durchgezogen.
Werkzeug	draw.io / diagrams.net. Abgabe als .drawio und als Bild-Export. Handskizzen, Screenshots von Whiteboards und Fotos werden nicht angenommen.
Umfang	4-6 Entitätstypen. Nicht weniger, nicht mehr.
Pflichtkonstrukte	≥ 1 N:M-Beziehung · ≥ 1 Generalisierung <i>oder</i> ≥ 1 schwacher Entitätstyp
Vollständigkeit	Alle Entitätstypen mit Attributen und Schlüsselkennzeichnung; alle Beziehungstypen benannt (Verb, nicht „hat“); alle Kardinalitäten in (min,max) angeschrieben.

Zusätzlich in docs/02-modell.md:

- **Zu jeder (min,max)-Angabe, die nicht (0,*) oder (1,1) ist: ein Satz, warum sie so ist.** Die Kardinalität ist eine Aussage über die Welt, nicht über das Diagramm. Wenn Sie schreiben (1,3), muss es einen Grund in der Miniwelt geben.
- Falls Sie eine **Generalisierung** verwenden: Ist sie *total/partiell* und *disjunkt/überlappend*? Warum?
- Falls Sie einen **schwachen Entitätstyp** verwenden: Was ist der Diskriminator, und warum reicht er allein nicht als Schlüssel?
- Falls Sie eine **ternäre Beziehung** erwägen: Begründen Sie, warum sie nicht in zwei binäre zerlegbar ist, oder zerlegen Sie sie.

Häufigster Fehler: Attribute, die eigentlich Entitätstypen sind (jede „Kategorie“, „Status“ oder „Typ“ als Freitextattribut ist ein Kandidat), und Beziehungen ohne Semantik („Kunde hat Bestellung“).

B3: Funktionale Abhängigkeiten

Abgabe: docs/03-fds.md · **Fällig:** M5

Dieser Punkt ist das Herzstück des Projekts. Er hat eine eigene Mindestschwelle in der Bewertung (siehe Bewertungsrubrik): Ohne mindestens die Hälfte der Punkte hier ist das Projekt nicht positiv.

Gefordert:

1. Eine Liste der funktionalen Abhängigkeiten Ihrer Miniwelt in der Form $\{A, B\} \rightarrow \{C, D\}$.

2. **Zu jeder FD eine Begründung aus der Anwendungssemantik**, also aus der Fachlogik Ihrer Miniwelt, nicht aus Ihren Testdaten.

✓ **Richtig:** „{Kursnummer} → {Kurstitel}“ gilt, weil das Studienrecht vorschreibt, dass eine Kursnummer über die Laufzeit eines Curriculums genau einem Titel zugeordnet bleibt.“

✗ **Falsch:** „{Kursnummer} → {Kurstitel}“ gilt, weil in meiner Tabelle jede Kursnummer nur einen Titel hat.“

3. **Mindestens eine FD, die Sie ausdrücklich verwerfen**, also eine, die in Ihren Testdaten scheinbar gilt, in der Miniwelt aber **nicht** gilt, mit Begründung und einem Gegenbeispiel-Datensatz.

Beispiel: „{Nachname} → {Abteilung}“ gilt in unseren Testdaten, weil es zufällig nur einen Huber gibt. In der Miniwelt gilt sie nicht: Zwei Personen mit gleichem Nachnamen können in verschiedenen Abteilungen arbeiten.“

4. Die **Kandidatenschlüssel** jeder Relation, mit Herleitung über die Attributhülle (kurz, aber nachvollziehbar).

Warum so viel Gewicht? Weil das die einzige Stelle im ganzen Datenbank-Curriculum ist, an der man eine Aussage über die *Welt* trifft und nicht über Syntax. Eine FD lässt sich aus einer Tabelle nie ablesen. Man kann aus einer Tabelleninstanz nur FDs *widerlegen*, nie *beweisen*. Wer das verstanden hat, normalisiert richtig. Wer es nicht verstanden hat, normalisiert zufällig.

B4: Relationales Schema in 3NF

Abgabe: docs/04-schema.md · **Fällig:** M5

Gefordert:

- Das relationale Schema in der Notation `Relation(PK, attr, attr, FK→Relation)`, Primärschlüssel unterstrichen oder markiert, Fremdschlüssel gekennzeichnet.
- Der Ableitungsweg vom EER-Diagramm zum Schema:** Wie wurde jede Beziehung umgesetzt (Fremdschlüssel / eigene Relation)? Wie die Generalisierung (eine Tabelle / Tabelle je Subtyp / Tabelle je Klasse)? Welche Alternative gab es, und warum diese?
- Der Nachweis der 3NF:** für jede Relation, welche FDs gelten, was ist der Kandidatenschlüssel, warum liegt keine partielle und keine transitive Abhängigkeit vor?
- Falls eine Relation nicht in BCNF ist:** Das ist zulässig. Benennen Sie die verletzende FD und erklären Sie, warum Sie **nicht** weiter zerlegen, typischerweise weil die Zerlegung nicht abhängigkeiterhaltend wäre.

Zielnormalform ist 3NF, nicht BCNF. 3NF garantiert Verlustfreiheit *und* Abhängigkeitserhaltung; BCNF garantiert nur Verlustfreiheit. In der Praxis ist die Abhängigkeitserhaltung meist wichtiger. Eine bewusst in 3NF belassene Relation mit sauberer Begründung bringt **mehr** Punkte als eine mechanisch bis BCNF zerlegte ohne.

B5: DDL mit kommentierten Constraints

Abgabe: sql/01-schema.sql · **Fällig:** M5

Setzen Sie Ihr Schema als PostgreSQL-DDL um.

Anforderung	Details
Eigenes Schema	Alle Objekte in einem benannten Schema (<code>CREATE SCHEMA <ihr_projekt>;</code>), nicht in <code>public</code> .
Datentypen	Bewusst gewählt. Insbesondere: <code>timestampz</code> statt <code>timestamp</code> bei allem, was einen Zeitpunkt meint; <code>numeric</code> statt <code>float</code> bei allem, was exakt sein muss (Geld, Mengen, Messwerte mit definierter Genauigkeit); Textlängen nur dort begrenzen, wo die Fachlogik es verlangt.
Primärschlüssel	Auf jeder Relation. Begründen Sie in <code>docs/04-schema.md</code> kurz, wo Sie einen natürlichen und wo einen künstlichen Schlüssel verwenden.
Fremdschlüssel	Vollständig, mit explizitem <code>ON DELETE / ON UPDATE</code> -Verhalten. Das Default-Verhalten <code>NO ACTION</code> ist eine zulässige Wahl, aber eine <i>Wahl</i> , die Sie treffen und begründen.
≥ 2 CHECK-Constraints	Die eine echte Fachregel durchsetzen, nicht nur <code>> 0</code> .
≥ 1 UNIQUE-Constraint	Über mehr als eine Spalte, der eine fachliche Eindeutigkeit abbildet.
<code>NOT NULL</code>	Überall dort, wo <code>NULL</code> fachlich keinen Sinn ergibt.
Kommentare	Jeder <code>CHECK</code> und jeder mehrspaltige <code>UNIQUE</code> bekommt eine Kommentarzeile darüber: <i>welche Fachregel</i> er durchsetzt, nicht was er technisch tut.

Reproduzierbarkeit: Das Skript muss auf einer leeren Datenbank fehlerfrei durchlaufen. Beginnen Sie mit `DROP SCHEMA IF EXISTS <ihr_projekt> CASCADE;`, damit ein Neuaufbau immer funktioniert. Das ersetzt händische Delete-Skripte.

```
-- Eine Anmeldung ist nur vor Kursbeginn möglich ($ Kursordnung Abs. 3).
-- Anmeldungen nach Kursbeginn werden fachlich abgelehnt.
ALTER TABLE kursverwaltung.anmeldung
  ADD CONSTRAINT chk_anmeldung_im_zeitfenster
  CHECK (angemeldet_am < beginnt_am);

-- Eine Person kann sich pro Kurstermin nur einmal anmelden.
-- Mehrfachbelegung desselben Termins ist fachlich ausgeschlossen.
ALTER TABLE kursverwaltung.anmeldung
  ADD CONSTRAINT uq_anmeldung_person_termin
  UNIQUE (person_id, termin_id);
```

B6: Testdaten an den Grenzen

Abgabe: `sql/02-daten.sql` · **Fällig:** M5

Gefordert: Genug Daten, um Ihre sechs Abfragen aus B7 sinnvoll zu testen, typischerweise 10–30 Zeilen pro Tabelle. **Nicht mehr.** Große generierte Datenmengen sind hier kein Qualitätsmerkmal (außer Sie wählen Extension E1).

Was zählt, ist nicht die Menge, sondern die Auswahl. Ihre Testdaten müssen mindestens enthalten:

- einen Fall, der **an der Grenze** eines `CHECK`-Constraints liegt (gerade noch erlaubt),
- einen Fall mit **NULL** in einem optionalen Attribut,
- mindestens einen Datensatz, der in einem `LEFT JOIN` aus B7 **keinen Partner** hat (sonst können Sie nicht zeigen, dass Sie den Unterschied zu `INNER JOIN` verstanden haben),
- bei N:M: eine Entität mit null, eine mit einer und eine mit mehreren Zuordnungen.

Zusätzlich: Legen Sie am Ende der Datei **drei auskommentierte INSERT-Statements** ab, die jeweils an einem anderen Constraint scheitern *würden*, mit Kommentar, welcher Constraint greift.

```
-- SCHEITERT an chk_anmeldung_im_zeitfenster: Anmeldung nach Kursbeginn
-- INSERT INTO kursverwaltung.anmeldung (person_id, termin_id, angemeldet_am)
-- VALUES (1, 4, '2027-03-01 10:00+01');
```

B7: Sechs SQL-Abfragen

Abgabe: sql/03-abfragen.sql · **Fällig:** M5

Schreiben Sie **genau sechs** Abfragen. Jede beantwortet eine fachliche Frage, die sich aus Ihren User Stories (B1) ergibt. Über allen sechs muss die folgende Abdeckung erfüllt sein:

#	Pflichtmerkmal	Anmerkung
1	Join über mindestens drei Tabellen	
2	GROUP BY mit HAVING	Das HAVING muss fachlich nötig sein, nicht dekorativ.
3	LEFT JOIN mit Begründung	Im Kommentar: Warum LEFT und nicht INNER? Was würde bei INNER verschwinden?
4	Unterabfrage oder CTE	Bei einer CTE: Warum ist sie hier lesbarer als die Alternative?
5	Bewusste NULL-Behandlung	Mindestens eine Abfrage muss sich explizit mit NULL auseinandersetzen (IS NULL, COALESCE, oder das Vermeiden der NOT IN-Falle). Kommentieren Sie, was ohne diese Behandlung passieren würde.
6	Frei	Eine Abfrage Ihrer Wahl, die eine Ihrer Top-3-User-Stories beantwortet.

```
-- -----
-- A3: Welche Kurse hatten im laufenden Semester keine einzige Anmeldung?
-- User Story: #2 (Priorität 2)
-- LEFT JOIN, weil genau die Kurse gesucht sind, für die KEIN Treffer in
-- anmeldung existiert. Mit INNER JOIN wäre das Ergebnis exakt die Menge
-- der Kurse, die wir NICHT suchen.
-- -----
SELECT k.kurs_id, k.titel
FROM kursverwaltung.kurs k
LEFT JOIN kursverwaltung.anmeldung a ON a.kurs_id = k.kurs_id
WHERE a.anmeldung_id IS NULL
AND k.semester = '2026W'
ORDER BY k.titel;
```

Nicht gefordert: Window Functions, rekursive CTEs, Performance-Tuning. Wer das einsetzen will, wählt Extension **E6**.

B8: verify.sql als Nachweis

Abgabe: sql/99-verify.sql · **Fällig:** M5 · **Punkte:** 10 (eigener Block)

Das ersetzt sämtliche Screenshots. Statt zu behaupten, dass Ihr Schema funktioniert, weisen Sie es maschinell nach.

Gefordert: Ein einzelnes SQL-Skript mit **mindestens sechs Prüfungen**. Jede Prüfung gibt genau eine Zeile mit einem Namen und PASS oder FAIL aus. Das Skript läuft nach 01-schema.sql und 02-daten.sql und muss **vollständig PASS** ergeben.

```
-- Prüfung 1: Referentielle Integrität, keine verwaisten Anmeldungen
SELECT 'P1 keine verwaisten Anmeldungen' AS pruefung,
      CASE WHEN COUNT(*) = 0 THEN 'PASS' ELSE 'FAIL' END AS ergebnis
FROM kursverwaltung.anmeldung a
LEFT JOIN kursverwaltung.kurs k ON k.kurs_id = a.kurs_id
WHERE k.kurs_id IS NULL;

-- Prüfung 2: Fachregel, kein Termin überschneidet sich im selben Raum
SELECT 'P2 keine Raumkollision' AS pruefung,
      CASE WHEN COUNT(*) = 0 THEN 'PASS' ELSE 'FAIL' END AS ergebnis
FROM kursverwaltung.termin t1
JOIN kursverwaltung.termin t2
  ON t1.raum_id = t2.raum_id
 AND t1.termin_id < t2.termin_id
 AND t1.beginnt_am < t2.endet_am
 AND t2.beginnt_am < t1.endet_am;
```

Abdeckung: mindestens je eine Prüfung aus jeder Kategorie

Kategorie	Beispiel
Referentielle Integrität	Keine verwaisten Fremdschlüssel
Fachregel	Eine Regel, die sich <i>nicht</i> als CHECK ausdrücken lässt (z. B. eine Überschneidung über mehrere Zeilen)
Erwartetes Aggregat	Eine Kennzahl aus Ihren Testdaten, die Sie unabhängig ausgerechnet haben
NULL-Semantik	Eine Prüfung, die zeigt, dass NULL dort steht, wo es hingehört, und nirgends sonst
Eindeutigkeit	Eine fachliche Eindeutigkeit, die über Ihren UNIQUE-Constraint hinausgeht
Frei	

Warum das mehr wert ist als Screenshots: Ein Screenshot zeigt, dass eine Abfrage einmal ein Ergebnis geliefert hat. `verify.sql` zeigt, dass Ihr Modell die Regeln Ihrer Miniwelt *tatsächlich durchsetzt*, und es lässt sich nach jeder Änderung erneut ausführen. Sie brauchen es außerdem in Teil 3 wieder.

B9: Minimaler Integrationsnachweis

Abgabe: `app/` (ca. 30 Zeilen) + `docs/09-integration.md` (ca. 10 Zeilen) · **Fällig:** M5

Ziel: Zeigen, dass Sie Ihre Datenbank aus einer Anwendung heraus korrekt ansprechen können, insbesondere mit **Prepared Statements**.

Variante A: JDBC-Snippet (Default). Ein einziges Java-Programm mit ca. 30 Zeilen, das (1) eine Verbindung aufbaut, (2) **eine** Ihrer Abfragen aus B7 mit einem **Prepared Statement** und mindestens einem Parameter ausführt, (3) das Ergebnis auf `stdout` ausgibt, (4) Ressourcen mit `try-with-resources` schließt.

Variante B: adaptierter REST-Endpunkt. Sie nehmen einen bestehenden Controller-Endpunkt aus dem Kurs-Codespace als Vorlage und passen ihn auf Ihr Schema an.

In `docs/09-integration.md` (ca. 10 Zeilen): Warum ein Prepared Statement und keine String-Konkatenation? Nennen Sie konkret, was mit *Ihrer* Abfrage passieren könnte, wenn Sie den Parameter direkt einsetzen.

Nicht gefordert: Schichtenarchitektur, DAO-Hierarchie, Tests, Dependency Injection.

B10: Entscheidungsprotokoll

Abgabe: docs/10-entscheidungen.md · **Fällig:** M5

Gefordert: Fünf Entscheidungen, die Sie im Projekt getroffen haben, im folgenden Format:

```
## Entscheidung 2: Generalisierung „Person“ als eine Tabelle statt drei

**Kontext:** Studierende und Vortragende haben fünf gemeinsame und je zwei eigene Attribute. Es gibt Personen, die beides sind.

**Entscheidung:** Eine Tabelle `person` mit einem Diskriminator und NULL-baren Subtyp-Attributen.

**Alternative:** Tabelle je Subtyp mit gemeinsamer `person`-Tabelle.

**Warum so:** Die Überlappung (Personen können beides sein) hätte bei Tabelle-je-Subtyp zu doppelten Einträgen oder einer zusätzlichen Rollen-Tabelle geführt. Bei nur zwei Subtyp-Attributen pro Rolle ist der NULL-Anteil vertretbar.

**Was dagegen spricht:** Die Subtyp-Attribute lassen sich nicht mit NOT NULL absichern. Wir fangen das über einen CHECK ab, der an den Diskriminator gebunden ist (siehe chk_person_rolleattribute).

**Verantwortlich:** A. Beispiel
```

Anforderung an die Auswahl: Es müssen **echte** Entscheidungen sein, also solche, bei denen es eine ernsthafte Alternative gab. „Wir haben PostgreSQL verwendet, weil es vorgegeben war“ ist keine Entscheidung.

Neu ab WS 2026/27, Feld „Verantwortlich“: Jede der fünf Entscheidungen wird einer Person namentlich zugeordnet (mindestens zwei pro Person). Das ist keine Schuldzuweisung, sondern die Grundlage dafür, dass Sie in Teil 3 zu *Ihren* Entscheidungen befragt werden können.

Kandidaten für gute Einträge: Umsetzung einer Generalisierung · natürlicher vs. künstlicher Primärschlüssel · Zerlegen oder Nicht-Zerlegen einer Relation · eine Kardinalität, die auch anders hätte ausfallen können · ON DELETE CASCADE vs. RESTRICT · eine Fachregel als CHECK vs. in der Anwendung · eine ternäre Beziehung zerlegt oder belassen.

B11: KI-Nutzungsprotokoll

Abgabe: docs/ai-usage.md · **Fällig:** M5

Verpflichtend, auch wenn Sie keine KI verwendet haben (dann ein Satz: „Keine KI-Werkzeuge verwendet.“).

Vorlage und vollständige Regeln: siehe **Bewertungsrubrik und KI-Regeln**, Kapitel 5. Kurzfassung:

KI-Nutzung ist erlaubt und erwünscht. Dokumentiert wird, **wo** Sie KI eingesetzt haben, **was** dabei herauskam und **was Sie damit gemacht haben**, insbesondere: was Sie verworfen haben und warum.

4. Teil 2: ENGINEERING EXTENSION (Gruppenleistung)

Umfang: ≈ 6–8 h pro Person · **Punkte:** 15 von 100

Sie wählen **genau eine** der sechs Optionen. Nicht zwei, nicht „eine Hauptoption und ein bisschen was anderes“. Die Extension ist die Stelle, an der Sie in *eine* Richtung in die Tiefe gehen.

Wahl bekanntgeben: bis Termin 5 (Fr, 04.12.2026), im Repository in `docs/20-extension.md` (erste Zeile: gewählte Option).

Für alle Optionen gilt das gleiche Abgabeformat:

Teil	Umfang	Inhalt
Artefakt	je Option verschieden	Das, was die Option verlangt (SQL, Code, Konfiguration, Messungen)
<code>docs/20-extension.md</code>	1-2 Seiten	Was war die Frage? Was haben Sie gemacht? Was ist herausgekommen, mit Zahlen oder Ausgaben? Was haben Sie daraus gelernt? Was würden Sie als Nächstes tun?

Bewertet wird nicht, ob das Ergebnis „gut“ ist, sondern ob Sie es erklären können. Eine Messung, die zeigt, dass Ihr Index *nichts* gebracht hat, mit einer korrekten Erklärung warum, ist voll punktefähig.

E1: Performance und Indizierung

Frage: Wo verbringt Ihre Datenbank ihre Zeit, und was ändert ein Index daran?

1. Testdaten auf ≥ 100.000 Zeilen in der größten Tabelle hochskalieren (`generate_series`, Skript in `sql/ext/e1-daten.sql`).
2. Zwei Ihrer Abfragen aus B7 mit `EXPLAIN (ANALYZE, BUFFERS)` messen: **vorher**.
3. Zwei Indizes anlegen, davon **mindestens einen zusammengesetzten**. Vor dem Anlegen aufschreiben, welche Verbesserung Sie erwarten und warum.
4. Erneut messen: **nachher**.
5. Auswerten: Plan-Änderung (Seq Scan $\rightarrow$ Index Scan?), Laufzeit, `Rows Removed by Filter`, `Buffers`.
6. **Beim zusammengesetzten Index:** Spaltenreihenfolge umdrehen und noch einmal messen. Erklären Sie den Unterschied.
7. Ein Fall, in dem der Index **nicht** genutzt wird, mit Erklärung.

Referenz im Unterricht: Foliensatz M5.3, Raindrops-Schema.

E2: Nebenläufigkeit und Transaktionen

Frage: Was passiert, wenn zwei Benutzer gleichzeitig dasselbe tun?

1. Eine Stelle in Ihrer Miniwelt identifizieren, an der eine Race Condition fachlich schadet (Doppelbuchung, Überbuchung eines Kontingents, Lost Update auf einem Zähler).
2. Das Problem **reproduzierbar herbeiführen:** zwei `psql`-Sitzungen, dokumentiert als Skriptpaar `sql/ext/e2-session-a.sql` / `e2-session-b.sql` mit Ausführungsreihenfolge.
3. Das Anomalie-Phänomen korrekt benennen (Lost Update / Dirty Read / Non-repeatable Read / Phantom / Write Skew).
4. **Drei Gegenmaßnahmen** erproben und vergleichen: Isolationsstufe erhöhen $\cdot$ `SELECT ... FOR UPDATE` $\cdot$ ein Constraint, der die Anomalie strukturell unmöglich macht.
5. Beurteilen: Welche Maßnahme ist für *Ihren* Fall die richtige, und was kostet sie?
6. **Pflichtbeobachtung:** Prüfen und dokumentieren Sie, ob `REPEATABLE READ` in PostgreSQL Phantom Reads verhindert, und was daraus für die Übertragbarkeit von Lehrbuchwissen folgt.

Referenz im Unterricht: Foliensatz M5.2, `DaoBase#runInRepeatableReadTransaction` im Kurs-Codespace.

E3: Verifikation und Datenqualität

Frage: Wie stellt man sicher, dass ein Datenbestand dauerhaft korrekt bleibt?

1. `verify.sql` (B8) auf **≥ 15 Prüfungen** ausbauen, gegliedert nach Kategorien.
2. Die Prüfungen so umbauen, dass das Skript **mit einem Fehlercode abbricht**, sobald eine Prüfung `FAIL` liefert (z. B. über `DO $$ ... RAISE EXCEPTION ... $$;`), also als echtes Testskript, nicht als Report.
3. Ein Shell-Skript `scripts/verify.sh`, das Schema, Daten und Prüfungen in einem Durchlauf gegen eine frische Datenbank ausführt.
4. **Mindestens drei mutierte Datensätze** in `sql/ext/e3-mutationen.sql`: gezielt fehlerhafte Daten, die jeweils eine andere Prüfung auslösen müssen. Nachweis, dass jede tatsächlich anschlägt (Mutation Testing im Kleinen).
5. Beurteilen: Welche Fehlerklasse in Ihrer Miniwelt würde Ihre Prüfsuite **nicht** finden?

Referenz im Unterricht: Foliensatz M8.1, `exercises/ai-review/` im Kurs-Codespace.

E4: Zugriffskontrolle

Frage: Wer darf was sehen, und wie setzt man das in der Datenbank durch, statt in der Anwendung?

1. Drei Rollen aus Ihrer Miniwelt ableiten (z. B. `leser`, `sachbearbeiter`, `admin`) mit je einem Satz, was sie fachlich dürfen.
2. Umsetzung mit `CREATE ROLE`, `GRANT`, `REVOKE` in `sql/ext/e4-rollen.sql`, nach dem Prinzip der minimalen Rechte.
3. **Mindestens eine View**, die eine Rolle sieht und eine andere nicht, als Werkzeug der Spaltensicherheit.
4. **Nachweis in `sql/ext/e4-verify.sql`**: Für jede Rolle mindestens ein Zugriff, der gelingen **muss**, und einer, der scheitern **muss**, mit `SET ROLE` reproduzierbar durchgespielt.
5. Beurteilen: Welche Ihrer fünf User Stories lässt sich mit reinem `GRANT` **nicht** absichern? Was bräuchte man dafür (Row Level Security, Anwendungslogik)?

Referenz im Unterricht: Foliensätze M5 (DCL) und M8.2 (RBAC).

E5: Analytics Engineering

Frage: Wie kommt man von einem OLTP-Schema zu einem Modell, mit dem man Fragen *auswertet* statt Vorgänge zu speichern?

1. Einen **Faktentyp** in Ihrer Miniwelt bestimmen und dessen **Grain** in einem Satz festlegen („Eine Zeile pro Anmeldung pro Termin“).
2. Ein kleines Stern-Schema entwerfen: 1 Faktentabelle + 2–3 Dimensionen, davon eine **Datumsdimension**.
3. Umsetzung als SQL-Views oder -Tabellen in `sql/ext/e5-marts.sql`, aufgeteilt in **Staging → Intermediate → Mart**.
4. **Zwei Auswertungsabfragen**, die auf dem Mart trivial sind und auf dem OLTP-Schema mühsam wären. Zeigen Sie beide Fassungen nebeneinander.
5. **Drei Datenqualitätsprüfungen** auf dem Mart (Eindeutigkeit des Grains, Wertebereich, Vollständigkeit gegenüber der Quelle).
6. Beurteilen: Welche Dimension in Ihrem Modell würde sich über die Zeit ändern? Wäre das SCD Type 1 oder Type 2, und warum?

 **Referenzmaterial:** Für Raindrops gibt es eine vollständige dbt-Umsetzung genau dieses Musters (Staging → Intermediate → Marts, Datums- und Sensordimension, Faktentabelle, Tests). Sie liegt im Arbeits-Repository der LV-Leitung und ist **nicht** Teil Ihrer Codespace-Vorlage; sie ist **kein Pflichtstoff**. Bei Interesse an E5 fragen Sie die LV-Leitung; sie kann den Aufbau im Termin zeigen.

dbt ist für diese Extension nicht erforderlich. Reines SQL genügt vollständig. Wer dbt verwenden möchte, darf das; es gibt dafür keine Zusatzpunkte.

Referenz im Unterricht: Foliensatz M5.4.

E6: Analytische SQL-Abfragen

Frage: Was kann SQL, das man mit `GROUP BY` allein nicht erreicht?

1. **Vier** Abfragen mit Window Functions auf Ihrem Schema, die je eine fachliche Frage beantworten. Abdeckung: mindestens eine Rangfolge (`RANK/DENSE_RANK`), mindestens ein Zeilenvergleich (`LAG/LEAD`), mindestens eine laufende Kennzahl mit explizitem Frame.
2. Zu **einer** dieser Abfragen: die Formulierung ohne Window Function (Self-Join oder korrelierte Unterabfrage) daneben stellen und beide mit `EXPLAIN ANALYZE` vergleichen.
3. **RANK vs. DENSE_RANK vs. ROW_NUMBER** an Ihren echten Daten mit Gleichständen zeigen und den Unterschied erklären, inklusive der Frage, welche Variante für Ihre fachliche Frage die richtige ist.
4. Eine Abfrage mit explizitem `ROWS BETWEEN ...` und Erklärung, was sich bei `RANGE` statt `ROWS` ändert (und warum das bei Gleichständen nicht dasselbe ist).
5. Beurteilen: An welcher Stelle kippt Ihre Abfrage von „lesbar“ in „nur noch mit Kommentar verständlich“?

Referenz im Unterricht: Foliensatz M5 (SQL vertieft).

5. Teil 3: ÄNDERUNGSSZENARIO (Einzelleistung)

Umfang: ≈ 4–5 h · **Punkte:** 20 von 100 · **Ausgabe:** letzter Präsenztermin (Fr, 11.12.2026) · **Abgabe:** mit M5

Dies ist der einzige Teil des Projekts, der individuell bewertet wird.

Jede Person bekommt ein **eigenes, persönlich zugeteiltes** Szenario. Die beiden Szenarien einer Gruppe treffen bewusst **unterschiedliche Teile** Ihres Modells.

5.1 Worum es geht

Ein Datenmodell ist nie fertig. Die eigentliche Prüfung eines Entwurfs ist nicht, ob er heute funktioniert, sondern was passiert, wenn sich morgen die Anforderung ändert. Genau das simulieren wir.

Sie bekommen **eine neue Anforderung an Ihre eigene Miniwelt** und beantworten schriftlich: Was ändert sich? Wo? Was bricht dabei? In welcher Reihenfolge muss man vorgehen?

Beispiele für Szenarien:

- „Ab sofort kann ein Kurs von zwei Vortragenden gemeinsam gehalten werden.“

- „Wir müssen nachvollziehen können, wie sich der Preis eines Artikels über die Zeit verändert hat.“
- „Ein Raum darf nun auch halbstündig gebucht werden statt nur in ganzen Terminen.“
- „Eine Anmeldung soll storniert werden können, ohne gelöscht zu werden.“

5.2 Der Typenkatalog

Damit Sie sich vorbereiten können, ist der **Typenkatalog offengelegt**. Ihr persönliches Szenario ist einer dieser zwölf Typen, angewandt auf Ihre konkrete Miniwelt:

#	Änderungstyp	Was dabei typischerweise passiert
T1	Kardinalität lockern (1:N → N:M)	Neue Beziehungsrelation, FK wandert, Abfragen liefern plötzlich Duplikate
T2	Kardinalität verschärfen (N:M → 1:N)	Datenmigration nötig, Mehrfachzuordnungen müssen aufgelöst werden
T3	Historisierung einführen	Gültigkeitszeitraum, Schlüssel wird zusammengesetzt, jede bestehende Abfrage braucht einen Zeitfilter
T4	Neues Attribut mit Fachregel	Neue Constraints, Altbestand verletzt sie evtl., NOT NULL mit Default vs. Migration
T5	Soft Delete / Storno statt Löschen	Statusattribut, alle Abfragen brauchen einen Filter, UNIQUE - Constraints müssen neu gedacht werden
T6	Neuer Subtyp in einer Generalisierung	Die Umsetzungsentscheidung aus B4 wird auf die Probe gestellt
T7	Attribut wird mehrwertig	Aus einer Spalte wird eine Relation, 1NF wird verletzt, wenn man es falsch macht
T8	Eine bestehende FD fällt weg	Normalform bricht, Zerlegung muss überdacht werden
T9	Feinere Granularität	Der Grain einer Relation ändert sich, Aggregate stimmen nicht mehr
T10	Zusätzliche Dimension in einer Beziehung	Binär → ternär, oder die Beziehung wird zum Entitätstyp
T11	Fachregel wandert in die Datenbank	Eine bisher nur in der Anwendung geprüfte Regel muss als Constraint ausdrückbar werden, oder nachweislich nicht
T12	Mandantenfähigkeit / neue Trennachse	Ein Schlüsselattribut kommt überall dazu, alle UNIQUE-Constraints müssen erweitert werden

Das Offenlegen des Katalogs ist Absicht. Die Arbeit liegt nicht darin, *welchen* Typ Sie bekommen, sondern darin, ihn auf *Ihr* Modell korrekt anzuwenden. Wer sich vorbereiten will, geht den Katalog vorher durch und überlegt zu jedem Typ, was er in der eigenen Miniwelt bedeuten würde. Das ist die beste Vorbereitung, die es gibt, und sie ist ausdrücklich erwünscht.

5.3 Was Sie abgeben

Zwei Dateien, beide mit Ihrem Nachnamen im Dateinamen:

(a) docs/30-aenderung-<nachname>.md : 1-2 Seiten, exakt diese sechs Abschnitte

Abschnitt	Inhalt	Umfang
1. Betroffene Modellteile	Welche Entitätstypen, Beziehungen und Attribute berührt die Anforderung? Was bleibt unberührt?	½ Seite
2. Änderung im konzeptuellen Modell	Welche Konstrukte kommen dazu/fallen weg/ändern ihre (min,max)? Skizze oder präzise Prosa, kein neues Gesamtdiagramm nötig.	½ Seite

Abschnitt	Inhalt	Umfang
3. Auswirkung auf FDs und Normalform	Welche FD kommt dazu, welche fällt weg? Bleibt 3NF erhalten? Wenn nein: Welche Zerlegung, und ist sie abhängigkeiterhaltend?	½ Seite
4. Bruchstellen	Welche Ihrer sechs Abfragen aus B7 liefern nach der Änderung falsche Ergebnisse, und warum? Welche Constraints greifen nicht mehr? Welche Testdaten werden ungültig? Mindestens zwei konkret benannte Bruchstellen mit Begründung.	½ Seite
5. Migrationsreihenfolge	Die Schritte in der Reihenfolge, in der sie ausgeführt werden müssen, mit einem Satz je Schritt, warum diese Reihenfolge. Wo wird Altbestand angefasst?	¼ Seite
6. Was dagegen spricht	Welchen Preis hat Ihre Lösung? Welche Alternative haben Sie verworfen, und warum?	¼ Seite

(b) `sql/30-aenderung-<nachname>.sql`: das DDL-Delta

Die ALTER- / CREATE- / UPDATE-Statements, die Ihre Änderung umsetzen.

Anforderungen:

- Muss **lauffähig** sein: nach `01-schema.sql` und `02-daten.sql` ausgeführt, muss das Skript fehlerfrei durchlaufen.
- Muss den **Altbestand** korrekt behandeln (also nicht nur `ALTER TABLE ... ADD COLUMN ... NOT NULL` ohne Plan für bestehende Zeilen).
- **Mindestens eine neue oder geänderte Prüfung** am Ende, im `PASS/FAIL`-Format aus B8, die zeigt, dass die neue Fachregel greift.
- Kommentiert nach demselben Muster wie B5.

Typischer Umfang: 15–40 Zeilen. Es geht nicht um Menge.

5.4 Bewertung: Überblick

Kriterium	Punkte
1: Betroffene Teile vollständig und richtig identifiziert	3
2: Modelländerung korrekt und begründet	4
3: FD-/Normalform-Analyse	5
4: Bruchstellen: konkret, benannt, richtig begründet	4
5 + 6: Migrationsreihenfolge und Abwägung	2
DDL-Delta lauffähig, Altbestand behandelt, Prüfung enthalten	2
Summe	20

Die vollständige Rubrik steht in der Bewertungsrubrik, Kapitel 4.

5.5 Regeln

- **Einzelarbeit.** Sie dürfen über Ihre Szenarien sprechen, aber jede Person schreibt ihr eigenes Dokument. Zwei Dokumente mit gleicher Argumentationsstruktur zu unterschiedlichen Szenarien fallen auf.
- **KI-Nutzung:** erlaubt, wie im ganzen Projekt, aber im Protokoll (B11) gesondert für Teil 3 auszuweisen. Und: Eine KI kennt Ihr Modell nicht. Antworten, die generisch über „Datenbankänderungen“ reden statt über *Ihre* fünf Relationen, sind hier sofort erkennbar und bringen fast keine Punkte. Abschnitt 4 (Bruchstellen) verlangt zwingend Bezug auf Ihre konkreten Abfragen aus B7.

- **Rückfragen zum Szenario** sind ausdrücklich erlaubt und erwünscht, per Mail an die LV-Leitung. Ein missverstandenes Szenario ist kein Prüfungsrisiko, das Sie tragen müssen.

6. Abgabeformat und Repository-Struktur

6.1 Struktur

Ein Git-Repository pro Gruppe, angelegt bis **Fr, 09.10.2026** (Link im eLearning). Geben Sie der LV-Leitung von Anfang an Zugriff: Schon M1 wird als Repository-Push abgegeben. Das Repository entsteht aus der Codespace-Vorlage der LVA (Anleitung: `00_Codespace_einrichten.pdf` in den Unterlagen zu Termin 1). Die Dateien der Vorlage (z. B. `src/`, `db-hr-schema-*`, `labs/`) bleiben daneben bestehen; bewertet werden die folgenden Pfade.

```
<gruppe>-dae5-projekt/
├── README.md                ← Thema, Gruppenmitglieder, Start in 5 Zeilen
├── docs/
│   ├── 01-anforderungen.md ← B1
│   ├── 02-modell.drawio     ← B2 (Quelle)
│   ├── 02-modell.png        ← B2 (Export)
│   ├── 02-modell.md         ← B2 (Begründungen)
│   ├── 03-fds.md            ← B3 🔑
│   ├── 04-schema.md         ← B4
│   ├── 09-integration.md    ← B9
│   ├── 10-entscheidungen.md ← B10
│   ├── 20-extension.md      ← Extension
│   ├── 30-aenderung-<nachname1>.md ← Teil 3, Person 1 🔑
│   ├── 30-aenderung-<nachname2>.md ← Teil 3, Person 2 🔑
│   └── ai-usage.md          ← B11
├── sql/
│   ├── 01-schema.sql        ← B5
│   ├── 02-daten.sql         ← B6
│   ├── 03-abfragen.sql      ← B7
│   ├── 30-aenderung-<nachname1>.sql ← Teil 3, Person 1
│   ├── 30-aenderung-<nachname2>.sql ← Teil 3, Person 2
│   ├── 99-verify.sql        ← B8 🔑
│   └── ext/                 ← Extension-Artefakte
├── app/                     ← B9 (ca. 30 Zeilen)
└── compose.yaml             ← optional, empfohlen
```

6.2 README

Fünf Zeilen genügen, aber diese fünf müssen stimmen:

```
# Kursverwaltung: DAE5-Projekt WS 2026/27
Gruppe 07 · Anna Beispiel (Szenario T3), Ben Muster (Szenario T7)
Extension: E2 (Nebenläufigkeit)

## Start
docker compose up -d
psql -f sql/01-schema.sql -f sql/02-daten.sql -f sql/99-verify.sql
```

6.3 Der Reproduzierbarkeits-Test

Vor der Abgabe: Klonen Sie Ihr eigenes Repository in ein leeres Verzeichnis (oder starten Sie einen neuen Codespace), legen Sie eine frische Datenbank an (im Codespace z. B. `CREATE DATABASE repro_test;`, verbunden mit `psql -h db`) und führen Sie aus:

`01-schema.sql` → `02-daten.sql` → `99-verify.sql` → `30-aenderung-<nachname>.sql`

Wenn `99-verify.sql` nicht vollständig `PASS` liefert oder ein Änderungsskript nicht durchläuft, ist das Projekt nicht abgabefertig.

Das ist kein formaler Akt. Ein nicht reproduzierbares Projekt kostet direkt Punkte in B8 und in Teil 3, und zwar ohne dass jemand Ihre Argumentation überhaupt liest.

6.4 Abgabe

- Repository bis zum Abgabetermin mit dem Git-Tag **abgabe-final** versehen (`git tag abgabe-final && git push --tags`).
- Prüfen, dass die LV-Leitung weiterhin Zugriff auf das Repository hat (eingrichtet seit dem Start, siehe 2.3).
- Commits nach dem Tag sind erlaubt, werden aber nicht bewertet.

Commit-Historie: Wir schauen sie an. Nicht, um die Commit-Anzahl zu bewerten, sondern weil sie bei Teil 3 die Zuordnung der Einzelleistungen stützt. Arbeiten Sie in kleinen, benannten Schritten und committen Sie Ihre eigenen Änderungsszenario-Dateien selbst.

7. Bewertung: Kurzüberblick

Die vollständigen Rubriken stehen in der **Bewertungsrubrik und KI-Regeln**. Hier nur das Gerüst:

Block	Punkte	Art
BUILD (B1-B7, B9)	55	Gruppe
Verifikation (B8)	10	Gruppe
Extension (E1-E6)	15	Gruppe
Änderungsszenario (Teil 3)	20	einzeln
Summe	100	

Drei Schwellen müssen gleichzeitig erfüllt sein:

Schwelle	Wert
Gesamtpunkte	≥ 50
Änderungsszenario (Teil 3)	≥ 8 von 20
B3 (funktionale Abhängigkeiten)	≥ 5 von 10

80 Punkte sind Gruppenpunkte, 20 individuell. Wie das zusammengerechnet wird (inklusive des Korrekturmechanismus bei stark abweichender Einzelleistung), steht in der Bewertungsrubrik, Kapitel 3.

8. KI-Nutzung: Kurzüberblick

KI-Nutzung ist in dieser Lehrveranstaltung ausdrücklich erlaubt und erwünscht.

Die vollständigen Regeln stehen in der Bewertungsrubrik, Kapitel 5. Die drei Sätze, die zählen:

1. **Sie dokumentieren die Nutzung** in `docs/ai-usage.md` (B11).
2. **Teil 3 fragt Sie zu Ihrem eigenen Modell. Wer eine Entscheidung nicht begründen kann, hat sie nicht getroffen.**
3. **Ein verworfener KI-Vorschlag mit Begründung ist in dieser LVA mehr wert als ein übernommener ohne.**

9. Häufige Fragen

Gibt es eine mündliche Prüfung?

Nein. Die LVA ist LV-immanent und wird vollständig schriftlich beurteilt. Der individuelle Anteil Ihrer Note entsteht über das Änderungsszenario (Teil 3).

Dürfen wir eine andere Datenbank als PostgreSQL verwenden?

Nein. Alle Beispiele, Constraints und Werkzeuge im Unterricht beziehen sich auf PostgreSQL 18. Extension E1 und E2 sind ohne PostgreSQL-spezifisches Verhalten nicht sinnvoll durchführbar.

Wir haben sieben Entitätstypen. Ist das ein Problem?

Ja, wahrscheinlich. Sieben Entitätstypen bedeuten meist, dass Sie etwas modellieren, das Sie in B1 hätten abgrenzen sollen. Kommen Sie damit in M1 oder M2, nicht im Jänner.

Muss unser Schema in BCNF sein?

Nein. Ziel ist 3NF. Eine bewusste Entscheidung gegen die BCNF-Zerlegung, korrekt begründet, ist besser bewertet als eine mechanische Zerlegung.

Dürfen wir Raindrops-Code kopieren?

Als Vorbild ja, als Abgabe nein. Wenn Sie Muster übernehmen (Idempotenz, Transaktionsklammer, DAO-Struktur), schreiben Sie in B10 dazu, was Sie übernommen und was Sie angepasst haben.

Können wir die Extension wechseln?

Bis Termin 5 (Fr, 04.12.2026) ja, danach nur nach Rücksprache.

Können wir uns beim Änderungsszenario gegenseitig helfen?

Reden ja, schreiben nein. Jede Person gibt ihr eigenes Dokument zu ihrem eigenen Szenario ab. Wenn Sie einander erklären, was der andere gerade versteht, ist das die beste Lernform, die es gibt, solange am Ende zwei eigenständige Texte stehen.

Was, wenn ich mein Szenario nicht verstehe?

Fragen Sie nach. Per Mail, jederzeit. Ein missverständenes Szenario ist ein Kommunikationsproblem, kein Prüfungsproblem.

Was, wenn ein Gruppenmitglied deutlich weniger beiträgt?

Sagen Sie es rechtzeitig. Unabhängig davon: Teil 3 ist individuell und deckt Beitragsungleichgewichte in der Regel auf (siehe Bewertungsrubrik, Kapitel 3.3).

Wie viel Zeit sollen wir wirklich investieren?

Die in Abschnitt 2.2 genannten Stunden. Mehr Aufwand ergibt in diesem Projekt nicht automatisch mehr Punkte. Die Rubriken belohnen Begründungsqualität, nicht Umfang.

10. Abgabe-Checkliste

Vor dem Setzen von `abgabe-final`:

Teil 1: BUILD

- ☐ B1: 5 priorisierte User Stories, 3 Abgrenzungen, 5 Glossarbegriffe
- ☐ B2: `.drawio` **und** Bild-Export, (min,max), 4–6 Entitätstypen, ≥ 1 N:M, ≥ 1 Generalisierung *oder* schwacher Entitätstyp, jede nicht-triviale Kardinalität begründet
- ☐ B3: alle FDs mit **semantischer** Begründung, ≥ 1 **verworfen**e FD mit Gegenbeispiel, Kandidatenschlüssel hergeleitet 
- ☐ B4: Ableitungsweg dokumentiert, 3NF nachgewiesen, BCNF-Abweichung (falls vorhanden) begründet

- ☐ B5: eigenes Schema, ≥ 2 CHECK, ≥ 1 mehrspaltiges UNIQUE, alle Constraints fachlich kommentiert, timestamp/numeric bewusst gewählt
- ☐ B6: Grenzfälle, NULL-Fall, LEFT-JOIN-Waise, 3 auskommentierte Negativ-Inserts
- ☐ B7: genau 6 Abfragen, alle Pflichtmerkmale abgedeckt, jede mit Kopfkomentar und User-Story-Bezug
- ☐ B8: 99-verify.sql mit ≥ 6 Prüfungen, alle Kategorien abgedeckt, **vollständig PASS** 🗝️
- ☐ B9: ca. 30 Zeilen, Prepared Statement mit Parameter, 10 Zeilen Begründung
- ☐ B10: 5 echte Entscheidungen mit Alternative, Gegenargument und **Feld „Verantwortlich“** (≥ 2 pro Person)
- ☐ B11: docs/ai-usage.md ausgefüllt, Teil 3 gesondert ausgewiesen

Teil 2: Extension

- ☐ Gewählte Option in der ersten Zeile von docs/20-extension.md
- ☐ Artefakte in sql/ext/
- ☐ docs/20-extension.md mit **Ergebnissen in Zahlen/Ausgaben**, nicht nur Beschreibung

Teil 3: Änderungsszenario (je Person eigenständig)

- ☐ docs/30-aenderung-<nachname>.md mit **allen sechs Abschnitten** 🗝️
- ☐ Abschnitt 4 nennt **mindestens zwei konkrete Bruchstellen** mit Bezug auf Ihre eigenen Abfragen aus B7
- ☐ sql/30-aenderung-<nachname>.sql läuft fehlerfrei nach 01 + 02 durch
- ☐ Altbestand im Änderungsskript korrekt behandelt
- ☐ Mindestens eine neue PASS/FAIL-Prüfung enthalten

Technik

- ☐ Frisches Klonen in leeres Verzeichnis getestet
- ☐ 01 → 02 → 99 → 30-* läuft fehlerfrei durch, 99 ist vollständig PASS
- ☐ README mit Thema, Namen, Szenario-Nummern, Extension und Startbefehl
- ☐ Zugriff für LV-Leitung eingerichtet
- ☐ Tag abgabe-final gesetzt und gepusht

Zum Schluss.

Dieses Projekt ist absichtlich schmal. Es verlangt nicht viele Dinge, sondern wenige Dinge gut begründet. Der Unterschied zwischen einem Projekt mit 60 und einem mit 90 Punkten liegt fast nie im Umfang. Er liegt darin, ob jemand sagen kann, *warum* etwas so ist und *was dagegen spricht*.

Das ist keine Prüfungstaktik. Das ist der eigentliche Beruf.

Fragen zum Projekt: im Termin, oder per Mail an die LV-Leitung. Bei inhaltlichen Fragen zur Modellierung: bitte mit Ihrem aktuellen Stand (Repository-Link oder Diagramm), nicht abstrakt.

Version 1.3 · Stand 30.09.2026 · gilt gemeinsam mit der Bewertungsrubrik und den KI-Regeln, Version 1.3